Tugas Besar 1

IF3170 Inteligensi Artifisial 2026/2027

Dosen Pengampu: Dr. Nur Ulfa Maulidevi, S.T, M.Sc.

Dr. Fariska Zakhralativa Ruskanda, S.T., M.T

Dr. Edward Ferdian S.T, B.Eng, M.Sc.



Kelompok (42) LiroLanguageModel:

I Gusti Ngurah Alit Dharma Yudha / 13524072

Jingglang Galih Rinenggan / 13524095

Rasyad Satyatma / 13524142

Deskripsi Permasalahan

Dalam kehidupan sehari-hari, permasalahan memuat suatu benda ke dalam kontainer sering kali muncul, umumnya tapi tidak terbatas pada industri logistik. Permasalahan dibuat lebih kompleks dengan munculnya beragam karakteristik yang dimiliki tiap benda, seperti kerapuhan benda, harga benda, dan dalam beberapa kasus yang membutuhkan properti spesifik ke domain tersebut, seperti paket yang harus dikirim sebelum terlambat. Masalah ini dapat dimodelkan menyerupai masalah *knapsack*. Masalah *knapsack* merupakan masalah klasik dalam komputasional dimana terdapat beberapa benda, masing-masing dengan harga dan berat yang harus dimasukkan ke dalam suatu kontainer, tanpa melebihi kapasitas maksimum kontainer tersebut. Pemodelan masalah *knapsack* dapat dimodifikasi lebih lanjut untuk menambahkan properti-properti lain untuk menyerupai kompleksitas dunia nyata.

Penyelesaian masalah *knapsack* umumnya memiliki beberapa pendekatan, salah satu pendekatan yang di-*explore* kali ini adalah penggunaan *local search* untuk menyelesaikan masalah *knapsack* dalam ruang tiga dimensi, dengan properti-properti benda tambahan seperti kerapuhan benda. Tidak seperti masalah *knapsack* biasa, beberapa batasan baru juga ditambahkan:

- Tidak boleh ada beberapa benda beririsan dalam kontainer.
- Tidak boleh ada benda yang melayang bebas.
- Tidak boleh ada benda yang rapuh tepat di bawah benda lainnya.
- Tidak boleh ada benda yang beririsan dengan dinding kontainer.

Setelah mendefinisikan batasan, benda dapat didefinisikan sebagai:

$$B_i = \{w_i, l_i, h_i, v_i, m_i, f_i, e_i\}$$

Dengan $I = \{1, 2, 3, \ldots, n\}$

Dan orientasi benda $O_i = Permutasi(w_i, l_i, h_i)$

Dimana:

w_i, l_i, h_i merupakan dimensi dari B_i

v_i merupakan harga

m_i merupakan berat

f_i menyatakan apakah B_i rapuh

e_i menyatakan *ETA (Estimated Time of Arrival)*

Serta kontainer sebagai:

$$C = \{W, L, H\}$$

Dimana W, L, H merupakan dimensi dari kontainer.

Solusi juga dapat didefinisikan sebagai subset dari I:

$$S \subseteq I$$

Dimana terdapat konfigurasi tiap benda dalam S yang memenuhi semua batasan

Meskipun demikian, menemukan solusi sering sekali tidak cukup. Terkadang, mencari solusi yang paling optimal masih dibutuhkan. Dalam pendekatan *local search*, kita dapat menggunakan *objective function* untuk menilai suatu konfigurasi (atau bisa disebut *state*), dan nilai tersebut digunakan untuk mengetahui seberapa optimal konfigurasi tersebut. Kali ini, *objective function* yang digunakan adalah:

$$max\ f(s) = \Sigma v_i$$

Yang memaksimalkan total nilai seluruh benda dalam kontainer.

Pembahasan

Dalam implementasi algoritma, ada beberapa kelas yang kami definisikan sebagai struktur data untuk merepresentasikan variabel dalam permasalahan ini yakni, truk dan paket/*package.* Dari kedua itu kami tambah juga kelas `State` untuk merepresentasikan keadaan truk setelah suatu generasi suksesor dalam algoritma. Seluruh implementasi kelas untuk komputasi algoritma berada dalam *directory* `src/kelasking.py`. Berikut *snippet* untuk implementasi setiap kelas.

```
class truck:
    def __init__(self, dimentions, max_capacity):
        self.dimentions = dimentions        # dict, ex.{"w":10, "l":20, "h":30}
        self.max_capacity = max_capacity    # int, satuan unit berat
        self.space = np.full((dimentions["h"], dimentions["l"], dimentions["w"]), None, dtype=object)
```

Kelas *truck* digunakan untuk menghasilkan initial state untuk algoritma Hill-Climbing dan Simulated Annealing. *Truck* juga digunakan untuk visualisasi.

```
class package:
    def __init__(self, id, dimentions, orientation, value, weight, is_fragile, ETA):
        self.id = id                        # int/str
        self.dimentions = dimentions        # dict, ex.{"w":1, "l":1, "h":1}
        self.orientation = orientation      # dict, ex.{"w":"x", "l":"y", "h":"z"}
        self.value = value                  # int/float
        self.weight = weight                # int/float
        self.is_fragile = is_fragile        # bool
        self.ETA = ETA                      # int
```

Kelas *package* digunakan untuk menyimpan state dari sebuah package. Terdapat 2 fungsi untuk kelas ini berupa `rotate` untuk merotasi package 90 derajat terhadap suatu sumbu dan `jason_susanto` untuk memudahkan serialize ke format JSON.

```
class state:
    def __init__(self, dimentions, inserted_pack, remaining_pack, max_weight):
        self.dimentions = dimentions
        # space[z][y][x] = package.id atau None (kosong); id boleh int/str
        self.space = np.full((dimentions["h"], dimentions["l"], dimentions["w"]), None, dtype=object)
        self.inserted_pack = inserted_pack   # packages dalam truk
        self.remaining_pack = remaining_pack # packages di luar truk
        self.max_weight = max_weight
        self.current_weight = sum(p.weight for p in inserted_pack)
        self.coords = {}  # package.id -> sudut kiri-bawah-belakang {"x","y","z"}
        self.orientations = {p.id: dict(p.orientation) for p in inserted_pack + remaining_pack}
        self.fragile_ids = frozenset(p.id for p in inserted_pack + remaining_pack if p.is_fragile)
```

Kelas *state* merupakan kelas paling utama yang digunakan dalam komputasi algoritma. Memiliki beberapa fungsi *private* seperti `place_package` yang memiliki *bounds checking.* Terdapat juga fungsi `objective` yang menghitung *value* dari sebuah *state*.

Dalam implementasi kami sebuah *state* sudah pasti *valid*, jadi tidak ada penalisasi *value* jika melanggar *constraint* yang disebutkan pada deskripsi permasalahan. Namun ada pengecualian dalam implementasi Genetic Algorithm.

A. Hill-Climbing Algorithm

a. Steepest Ascent

Mengikuti konsep utama dari HC-Steepest Ascent, algoritma akan memulai dengan initial state yang random. Setelah itu akan di-*generate* seluruh suksesor dari state tersebut. Dari seluruh state pilih yang memiliki value tertinggi menggunakan objective function. Kemudian jika value tersebut lebih besar dari value state saat ini, lanjutkan iterasi selanjutnya dengan state suksesor sebagai state current. Jika value <= value state saat ini algoritma berhenti. Berikut snippet implementasinya,

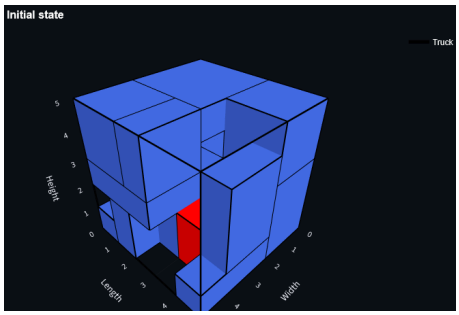
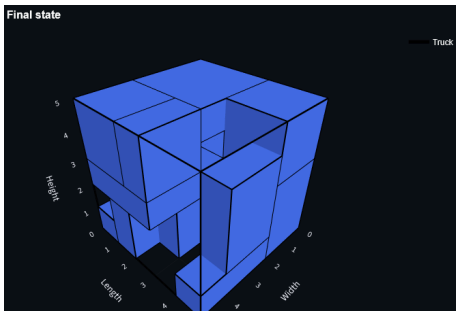
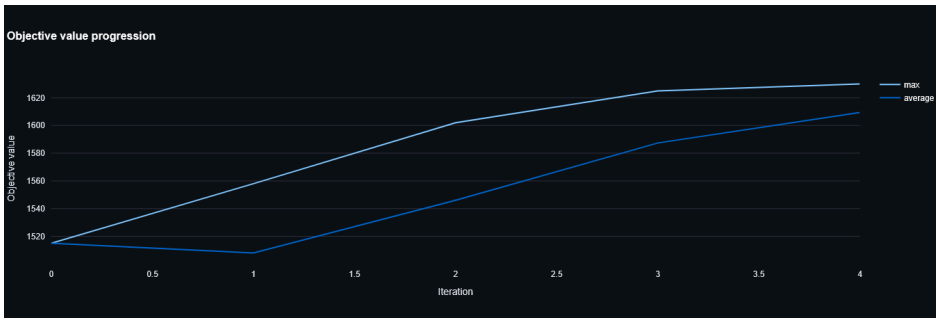
```
def neighbors(cur):
    ins = sorted(cur.inserted_pack, key=lambda p: p.id)
    outs = sorted(cur.remaining_pack, key=lambda p: p.id)
    d = cur.dimentions
    for i, a in enumerate(ins):
        for b in ins[i + 1:]:
            s = _apply(cur, [("remove", a), ("remove", b),
                             ("place", a, a.orientation, cur.coords[b.id]),
                             ("place", b, b.orientation, cur.coords[a.id])])
            if s is not None:
                yield s
    for a in ins:
        for b in outs:
            s = _apply(cur, [("remove", a), ("place", b, b.orientation, cur.coords[a.id])])
            if s is not None:
                yield s
    for p in ins:
        eff = eff_dims(p.dimentions, p.orientation)
        c0 = cur.coords[p.id]
        for z in range(d["h"] - eff["z"] + 1):
            for y in range(d["l"] - eff["y"] + 1):
                for x in range(d["w"] - eff["x"] + 1):
                    if x == c0["x"] and y == c0["y"] and z == c0["z"]:
                        continue
                    s = _apply(cur, [("remove", p), ("place", p, p.orientation, {"x": x, "y": y, "z": z})])
                    if s is not None:
                        yield s
    for p in ins:
        c0 = cur.coords[p.id]
        for axis in AXES:
            orientation = rotated_orientation(p.orientation, axis)
            eff = eff_dims(p.dimentions, orientation)
            if (c0["x"] + eff["x"] > d["w"] or c0["y"] + eff["y"] > d["l"]
                    or c0["z"] + eff["z"] > d["h"]):
                continue
            s = _apply(cur, [("remove", p), ("place", p, orientation, dict(c0))])
            if s is not None:
                yield s
```

```
def steepest_ascent(initial):
    cur = initial
    history = [cur.objective()]
    average_history = [cur.objective()]
    iterations = 0
    while True:
        best, best_val = None, cur.objective()
        values = []
        for s in neighbors(cur):
            v = s.objective()
            values.append(v)
            if v > best_val:
                best, best_val = s, v
        if best is None:
            return cur, history, average_history, iterations
        cur, iterations = best, iterations + 1
        history.append(best_val)
        average_history.append(sum(values) / len(values))
```

Algoritma menghasilkan final state, list value state terbaik untuk setiap iterasi, rata-rata value state untuk setiap iterasi, dan jumlah iterasi.

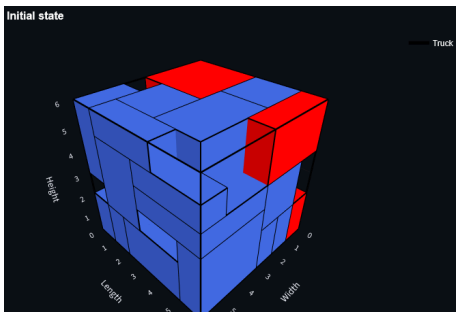
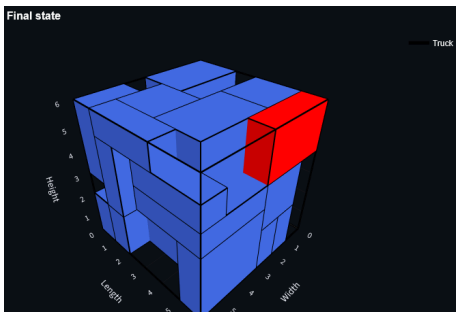
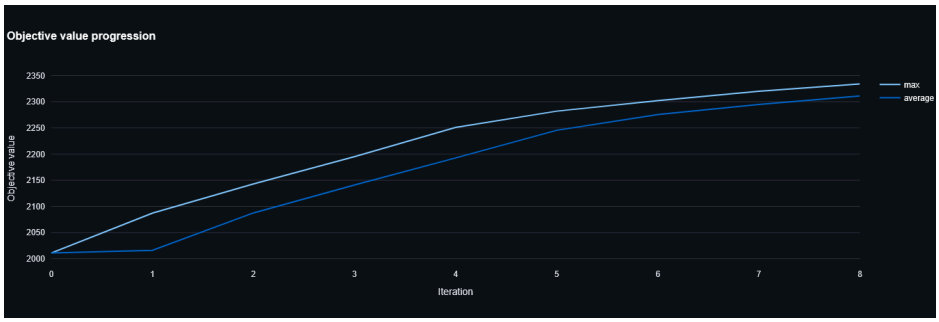
Testing

1. Test Case 1

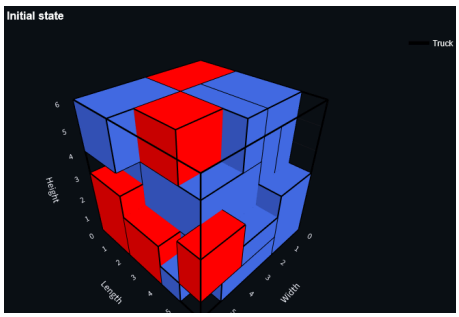
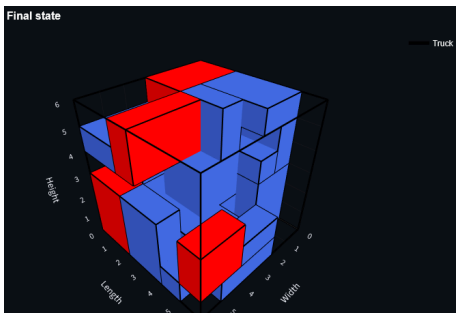
Initial State	Final State
	
Objective Function Plot	
	
Iteration Count	4
Processing Time	0.5529s
Initial Value	1515
Final Value	1630

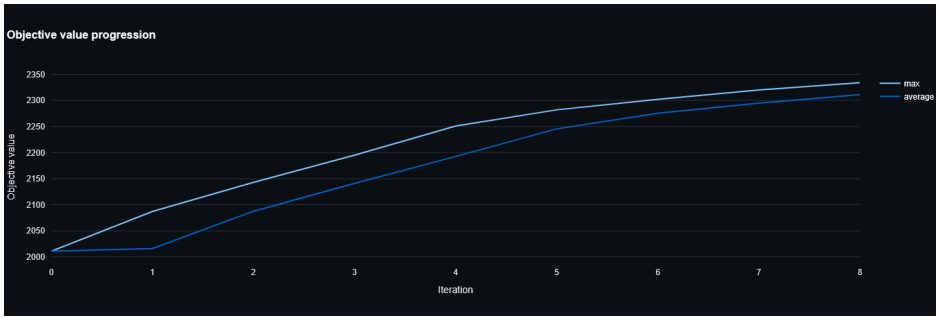
2. Test Case 2

Initial State	Final State

	
Objective Function Plot	
	
Iteration Count	8
Processing Time	5.0011s
Initial Value	8
Final Value	2011

3. Test Case 3

Initial State	Final State
	
Objective Function Plot	



Iteration Count	12
Processing Time	6.4984s
Initial Value	4773
Final Value	5318

Analisis

Algoritma Hill-Climbing Steepest Ascent secara konsisten mampu menemukan solusi yang lebih optimal dibanding initial state. Waktu pencarian tidak jauh berbeda untuk tiap test-case, dan linear naik mengikuti jumlah *package* dalam tiap test-case.

B. Simulated Annealing

Konsep dari Simulated Annealing ini seperti Stochastic Hill-Climbing dengan diperbolehkannya beberapa *move downhill.* Bermula sama seperti Hill-Climbing, pembuatan initial state yang random. Kemudian akan di generate hanya satu suksesor dari state, implementasinya menyerupai generasi suksesor pada Hill-Climbing tetapi akan di-generasi 1 state secara random, berikut penjelasan implementasinya,

```
def hello_neighbor(cur):
    ins = sorted(cur.inserted_pack, key=lambda p: p.id)
    outs = sorted(cur.remaining_pack, key=lambda p: p.id)
    if not ins:
        return None

    d = cur.dimentions
    actions = ["move", "rotate"]
    if len(ins) >= 2:
        actions.append("swap_inside")
    if outs:
        actions.append("swap_outside")
```

Fungsi akan menerima state, ada definisi 4 jenis aksi yang dapat dilakukan yaitu “move” (memindahkan *package* yang berada di dalam truk ke koordinat lain), “rotate” (merotasi *package* 90 derajat terhadap suatu sumbu), “swap_inside” (menukar posisi 2 *package* yang saat ini berada dalam truk), “swap_outside” (menukar sebuah package yang berada di dalam dengan yang berada di luar truk).

```
for _ in range(100):
    action = random.choice(actions)

    if action == "swap_inside":
        a, b = random.sample(ins, 2)
        candidate = _apply(cur, [
            ("remove", a), ("remove", b),
            ("place", a, cur.orientations[a.id], cur.coords[b.id]),
            ("place", b, cur.orientations[b.id], cur.coords[a.id]),
        ])
```

Dilakukan loop sebanyak 100 untuk antisipasi kasus hasil state tidak valid yang menghasilkan `None`.

```
        elif action == "swap_outside":
            a = random.choice(ins)
            b = random.choice(outs)
            candidate = _apply(cur, [
                ("remove", a),
                ("place", b, cur.orientations[b.id], cur.coords[a.id]),
            ])
```

```
        elif action == "move":
            p = random.choice(ins)
            orientation = cur.orientations[p.id]
            eff = eff_dims(p.dimentions, orientation)
            c0 = {
                "x": random.randrange(d["w"] - eff["x"] + 1),
                "y": random.randrange(d["l"] - eff["y"] + 1),
                "z": random.randrange(d["h"] - eff["z"] + 1),
            }
            if c0 == cur.coords[p.id]:
                continue
            candidate = _apply(cur, [
                ("remove", p),
                ("place", p, orientation, c0),
            ])
```

```
        else:
            p = random.choice(ins)
            axis = random.choice(AXES)
            orientation = rotated_orientation(cur.orientations[p.id], axis)
            c0 = cur.coords[p.id]
            eff = eff_dims(p.dimentions, orientation)
            if (c0["x"] + eff["x"] > d["w"]
                    or c0["y"] + eff["y"] > d["l"]
                    or c0["z"] + eff["z"] > d["h"]):
                continue
            candidate = _apply(cur, [
                ("remove", p),
                ("place", p, orientation, dict(c0)),
            ])

        if candidate is not None:
            return candidate

    return None
```

Algoritma melakukan loop dengan pengecekan, apakah temperatur masih di atas threshold dan apakah jumlah iterasi <= 10000 (dibatasi hardcoded). Dalam setiap iterasi, generasi suksesor, jika value dari suksesor lebih besar dari state saat ini ambil suksesor. Jika value suksesor lebih kecil generasi angka (0..1) random, jika angka tersebut di dalam threshold probability yang dihitung untuk setiap suksesor, ambil suksesor. Jika angka >= probability, lanjut iterasi berikutnya. Berikut snippet implementasinya,

```python
def simulated_annealing(initial, temp=100.0, cooling_rate=0.995, minimum_temp=0.01):
    cur = initial
    history = [cur.objective()]
    average_history = [cur.objective()]
    probability_history = []
    stuck_history = []
    iterations = 0

    while temp > minimum_temp and iterations <= 10000:
        succ = hello_neighbor(cur)
        if succ is None:
            break
        curr_value = cur.objective()
        succ_value = succ.objective()
        stuck_history.append(succ_value <= curr_value)
        delta_e = succ_value - curr_value
        probability = math.exp(delta_e / temp)
        average_history.append(succ_value)

        if delta_e > 0:
            cur = succ
        else:
            if random.random() < probability:
                cur = succ

        history.append(cur.objective())
        probability_history.append(probability)
        temp, iterations = temp * cooling_rate, iterations + 1

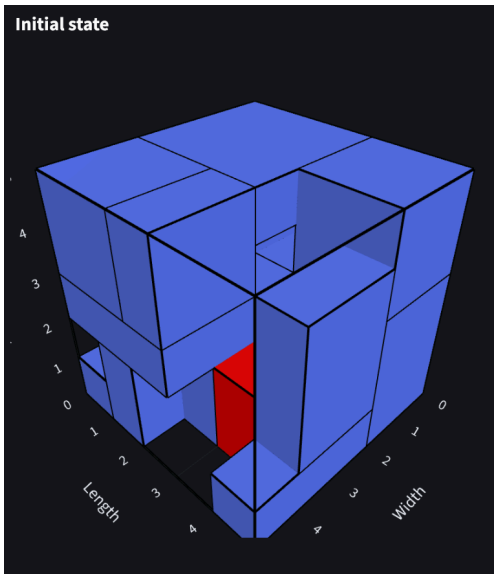
    return cur, history, average_history, probability_history, stuck_history, iterations
```

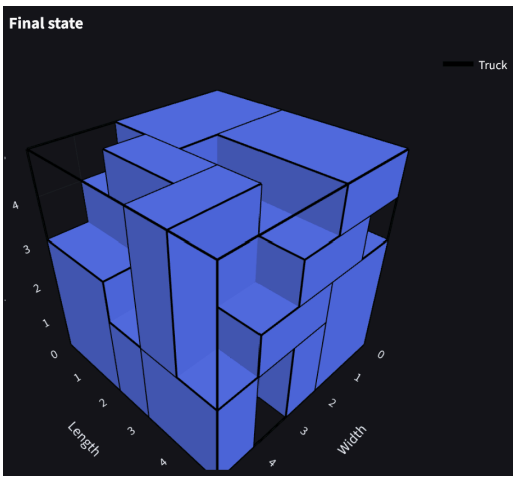
Akan menghasilkan final state, list state value untuk setiap iterasi, list probability untuk setiap iterasi, list "stuck" pada local optima atau tidak untuk setiap iterasi, dan total iterasi. Untuk "stuck" pada local optima ini didefinisikan sebagai jika value suksesor lebih kecil dari value *current state*.

Testing

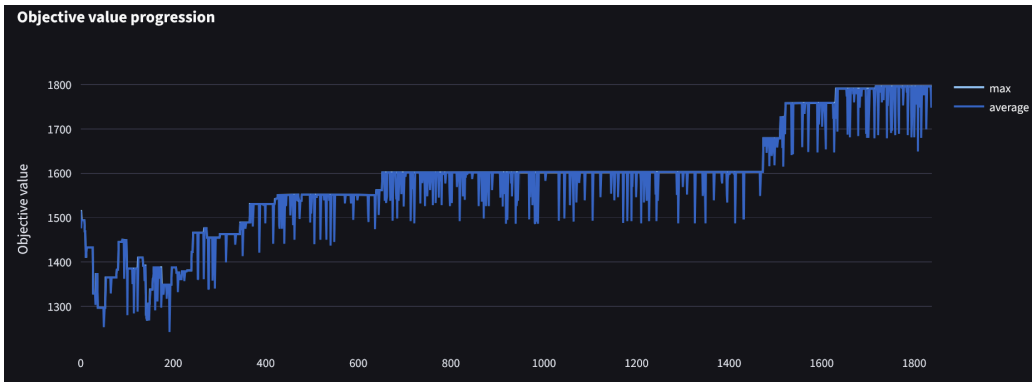
1. **Test Case 1**

Initial State	Final State

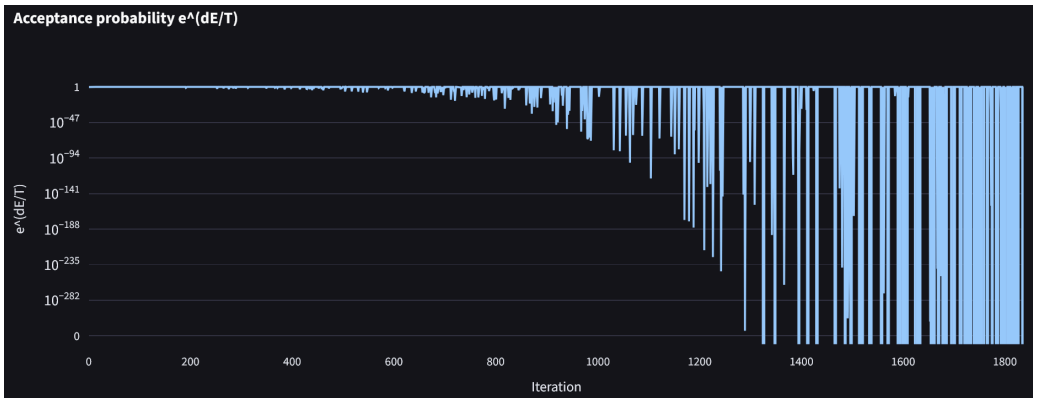




Objective Function Plot



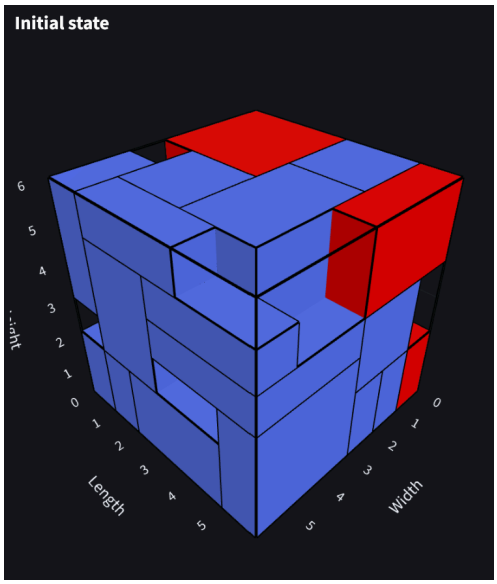
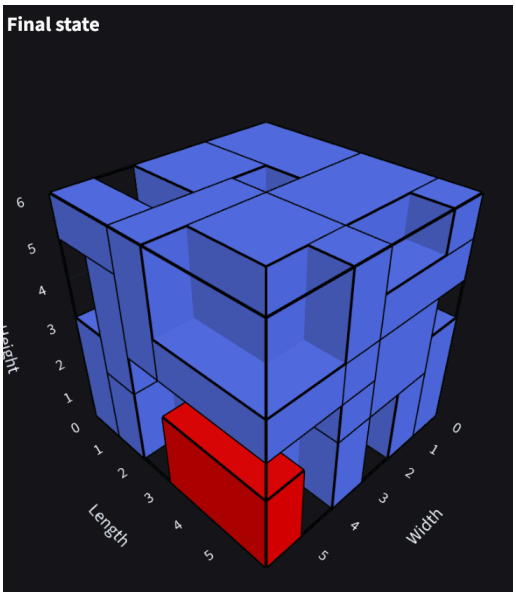
Probability



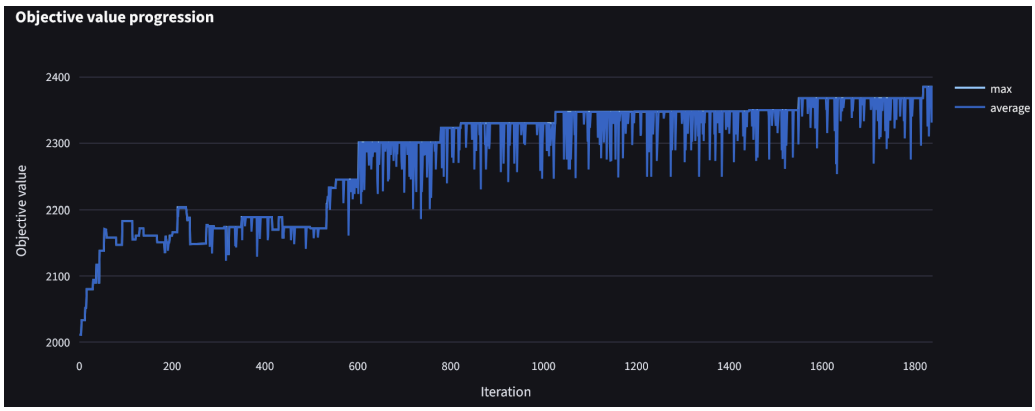
Iteration Count	1838

Processing Time	0.4606s
Initial Value	1515
Final Value	1796
Stuck Frequency	1802

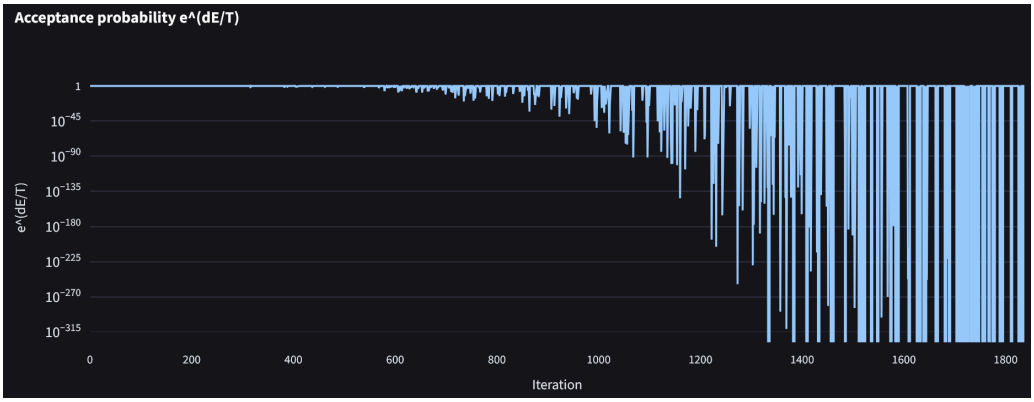
2. Test Case 2

Initial State	Final State
	

Objective Function Plot

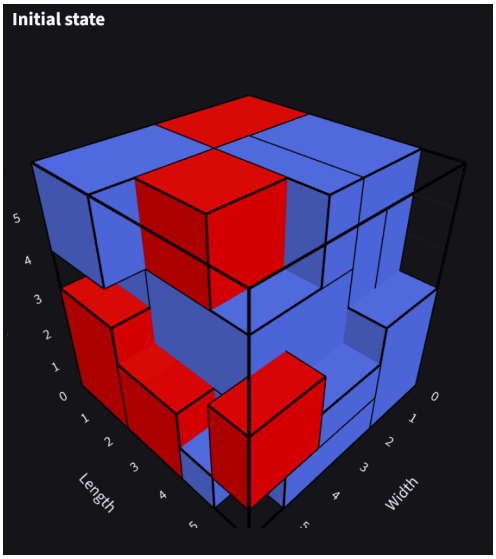
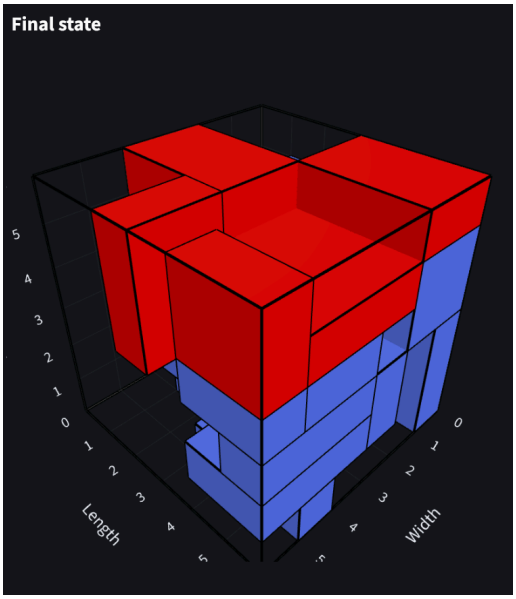


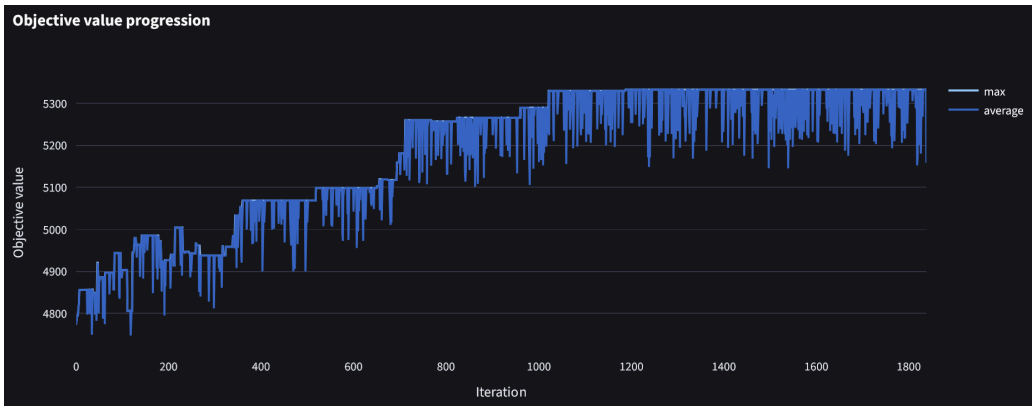
Probability



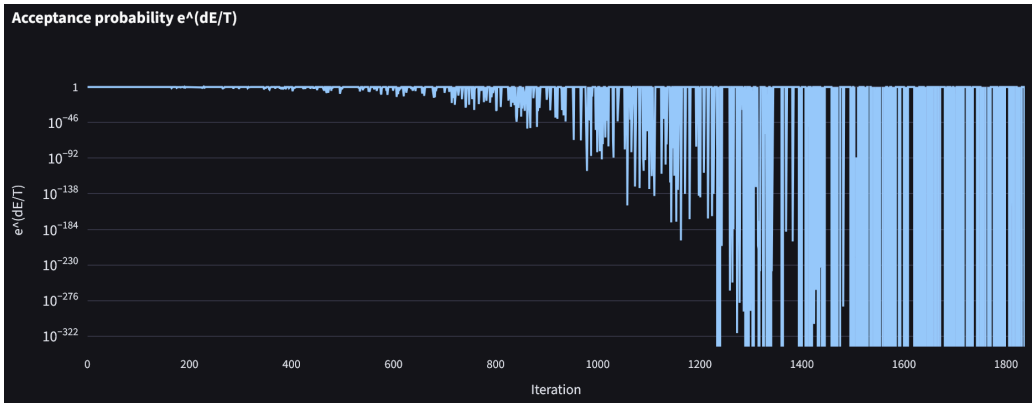
Iteration Count	1838
Processing Time	1.3158s
Initial Value	2011
Final Value	2385
Stuck Frequency	1804

3. Test Case 3

Initial State	Final State
	
Objective Function Plot	



Probability



Iteration Count	1838
Processing Time	1.0976s
Initial Value	4773
Final Value	5333
Stuck Frequency	1803

Analisis

Hasil dari Simulated Annealing lebih besar untuk beberapa test case. Meskipun berjalan dalam ribuan iterasi, waktu pemrosesan tergolong sangat singkat dan stabil karena hanya mengevaluasi satu suksesor acak pada setiap langkahnya.

C. Genetic Algorithm

Dalam implementasi kami *chromosome* direpresentasikan sebagai *state*, dan *gene* dalam *chromosome* berupa kondisi *package* dalam *state* tersebut. Sesuai dengan konsep Genetic Algorithm, implementasi kami dimulai dengan membuat sejumlah N yaitu ukuran populasi. Berikut implementasinya,

```
def _orientations(pkg):
    base = dict(pkg.orientation)
    return [
        base,
        rotated_orientation(base, "x"),
        rotated_orientation(base, "y"),
        rotated_orientation(base, "z"),
        rotated_orientation(rotated_orientation(base, "z"), "x"),
        rotated_orientation(rotated_orientation(base, "z"), "y"),
    ]

def _place_at_any_valid_position(chromosome, pkg, orientation):
    dimensions = chromosome.dimentions
    for z in range(dimensions["h"]):
        for y in range(dimensions["l"]):
            for x in range(dimensions["w"]):
                if chromosome.place_package(
                    pkg, {"x": x, "y": y, "z": z}, orientation
                ):
                    return True
    return False

def create_initial_population(trukkun, packages, N_population) -> list:
    population = []
    for _ in range(N_population):
        chromosome = state(trukkun.dimentions, [], [copy.copy(pkg) for pkg in packages], trukkun.max_capacity)
        try_packages = chromosome.remaining_pack.copy()
        random.shuffle(try_packages)
        for pkg in try_packages:
            orientations = _orientations(pkg)
            random.shuffle(orientations)
            for orientation in orientations:
                if _place_at_any_valid_position(chromosome, pkg, orientation):
                    break
        population.append(chromosome)
    return population
```

Kurang lebih sama dengan generasi initial state pada HC dan Simulated Annealing, hanya dilakukan N kali. Kemudian dalam loop sebanyak jumlah generasi, pertama dilakukan perhitungan fitness untuk setiap chromosome dalam populasi. Fitness dihitung dengan jumlah value suatu state dikurangi dengan penalisasi, berikut implementasinya,

```python
# basiclaly objective with penalty subtraction if state is not valid
def fitness(chromosome):
    value = chromosome.objective()
    violations = _constraint_violations(chromosome)
    if violations == 0:
        return value
    penalty = sum(abs(pkg.value) for pkg in chromosome.inserted_pack) + 1
    return value - penalty * violations
```

```python
# state validiy checking
def _constraint_violations(chromosome):
    violations = 0
    inserted_ids = [pkg.id for pkg in chromosome.inserted_pack]
    remaining_ids = [pkg.id for pkg in chromosome.remaining_pack]
    all_ids = inserted_ids + remaining_ids

    violations += len(all_ids) - len(set(all_ids))
    if chromosome.current_weight != sum(
        pkg.weight for pkg in chromosome.inserted_pack
    ):
        violations += 1
    if chromosome.current_weight > chromosome.max_weight:
        violations += 1

    occupied = set()
    for pkg in chromosome.inserted_pack:
        if pkg.id not in chromosome.coords or pkg.id not in chromosome.orientations:
            violations += 1
            continue
        dimensions = eff_dims(
            pkg.dimentions, chromosome.orientations[pkg.id]
        )
        coords = chromosome.coords[pkg.id]
        x0, y0, z0 = coords["x"], coords["y"], coords["z"]
        if (
            x0 < 0
            or y0 < 0
            or z0 < 0
            or x0 + dimensions["x"] > chromosome.dimentions["w"]
            or y0 + dimensions["y"] > chromosome.dimentions["l"]
            or z0 + dimensions["z"] > chromosome.dimentions["h"]
        ):
            violations += 1
            continue
```

```python
        for z in range(z0, z0 + dimensions["z"]):
            for y in range(y0, y0 + dimensions["y"]):
                for x in range(x0, x0 + dimensions["x"]):
                    cell = (x, y, z)
                    if cell in occupied or chromosome.space[z, y, x] != pkg.id:
                        violations += 1
                    occupied.add(cell)

    if not chromosome.is_valid():
        violations += 1
    return violations
```

Violation di sini terjadi jika suatu *package overlap* dengan *package* lain, ketika *package* berada di luar *bounds* truk, dan ketika melebihi batas weight untuk truk.

Setelah perhitungan fitness, kita akan membentuk populasi untuk generasi sebelumnya. Loop sebanyak ukuran populasi, pilih 2 parent secara random menggunakan roulette wheel, berikut implementasi pemilihan sebuah parent,

```python
# rng select parent
def _select(population, scores):
    minimum = min(scores)
    weights = [score - minimum for score in scores]

    if sum(weights) == 0:
        weights = [1] * len(population)

    total_weight = sum(weights)
    rng_num = random.random() * total_weight
    cumulative_weight = 0

    for chromosome, weight in zip(population, weights):
        cumulative_weight += weight
        if rng_num < cumulative_weight:
            return chromosome

    return population[-1]
```

Setelah itu kedua parent akan dilakukan cross untuk menghasilkan child. Cross dilakukan dengan mengambil posisi *package* dari parent a dan b, kemudian memilih secara random yang akan dimasukan ke dalam child state. Jika posisi tidak valid akan dipindah hingga state valid. Berikut implementasinya,

```python
def kawin(chromosome_a, chromosome_b):
    # Cross package placements from both parents. If a placement conflicts
    # with an earlier inherited placement, keep it and let fitness penalize it.
    parent_packages = _package_by_id(chromosome_a)
    parent_packages.update(_package_by_id(chromosome_b))
    child = state(chromosome_a.dimentions, [], [copy.copy(pkg) for pkg in parent_packages.values()], chromosome_a.max_weight)
    child_packages = _package_by_id(child)

    package_ids = list(parent_packages)
    random.shuffle(package_ids)
    package_ids.sort(key=lambda package_id: parent_packages[package_id].value, reverse=True)

    for package_id in package_ids:
        candidates = []
        if package_id in chromosome_a.coords:
            candidates.extend(_placement_candidates(chromosome_a, package_id))
        if package_id in chromosome_b.coords:
            candidates.extend(_placement_candidates(chromosome_b, package_id))
        random.shuffle(candidates)

        placed = False
        for pkg, coords, orientation in candidates:
            child_pkg = child_packages[package_id]
            if child.place_package(child_pkg, coords, orientation):
                placed = True
                break
            if _force_place(child, child_pkg, coords, orientation):
                placed = True
                break
        if not placed:
            continue
    return child
```

Setelah cross akan ada chance mutasi pada child sebesar 0.05, mutation secara random mengubah suatu configurasi sebuah package dalam sebuah chromoosome, berikut implementasinya,

```python
def _rebuild_with_mutation(chromosome, package_id):
    packages = _package_by_id(chromosome)
    rebuilt = state(chromosome.dimentions, [], [copy.copy(pkg) for pkg in packages.values()], chromosome.max_weight)
    rebuilt_packages = _package_by_id(rebuilt)
    order = [pkg.id for pkg in chromosome.inserted_pack if pkg.id != package_id]
    order.append(package_id)
    random.shuffle(order)

    for current_id in order:
        pkg = rebuilt_packages[current_id]
        orientations = _orientations(pkg)
        if current_id != package_id and current_id in chromosome.orientations:
            inherited = chromosome.orientations[current_id]
            orientations.insert(0, inherited)
        random.shuffle(orientations)
        for orientation in orientations:
            if _place_at_any_valid_position(rebuilt, pkg, orientation):
                break
    return rebuilt

def mutate(chromosome):
    if not chromosome.inserted_pack:
        return chromosome.copy()
    package_id = random.choice(chromosome.inserted_pack).id
    return _rebuild_with_mutation(chromosome, package_id)
```

Setelah itu lakukan sampai generasi berikutnya telah tergenerasi. Berikut implementasi Genetic Algorithm yang barusan dijelaskan,

```python
def genetic_algorithm(initial, population_size=50, generations=100, mutation_rate=0.05):
    trukkun = truck(initial.dimentions, initial.max_weight)
    packages = initial.inserted_pack + initial.remaining_pack
    population = create_initial_population(trukkun, packages, population_size)
    population[0] = initial.copy()

    max_objective_history = [max(fitness(chromosome) for chromosome in population)]
    average_objective_history = [sum(fitness(chromosome) for chromosome in population) / population_size]

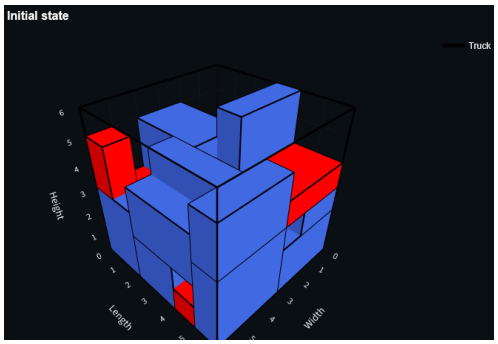
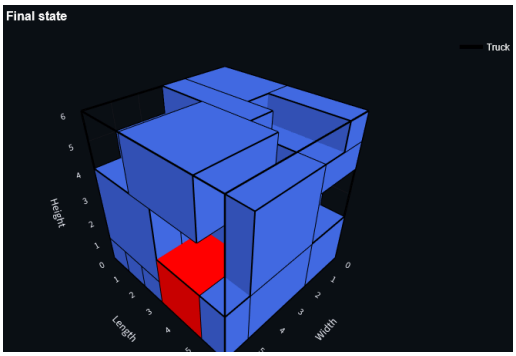
    for _ in range(generations):
        scores = [fitness(chromosome) for chromosome in population]
        new_population = [population[scores.index(max(scores))].copy()]
        while len(new_population) < population_size:
            parent_a = _select(population, scores)
            parent_b = _select(population, scores)
            child = kawin(parent_a, parent_b)
            if random.random() < mutation_rate:
                child = mutate(child)
            new_population.append(child)
        population = new_population
        objectives = [fitness(chromosome) for chromosome in population]
        max_objective_history.append(max(objectives))
        average_objective_history.append(sum(objectives) / population_size)

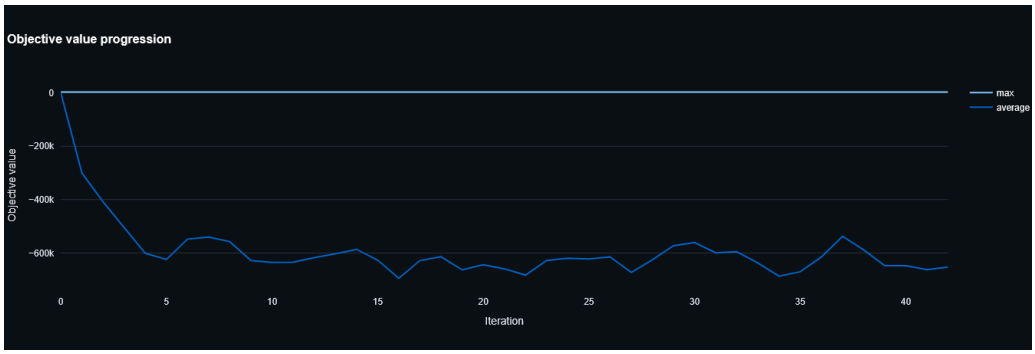
    final = max(population, key=fitness)
    return final, max_objective_history, average_objective_history, population_size, generations
```

Fungsi ini menghasilkan final state, list value maksimum dari setiap generasi, list value rata-rata dari setiap generasi, ukuran populasi, dan jumlah generasi.

Testing

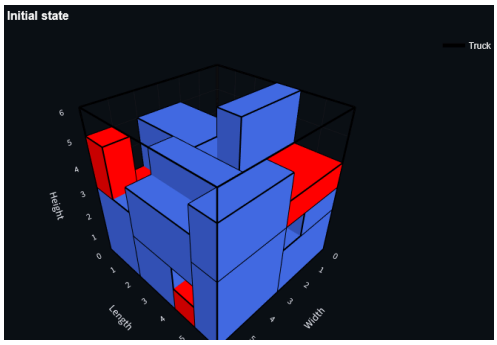
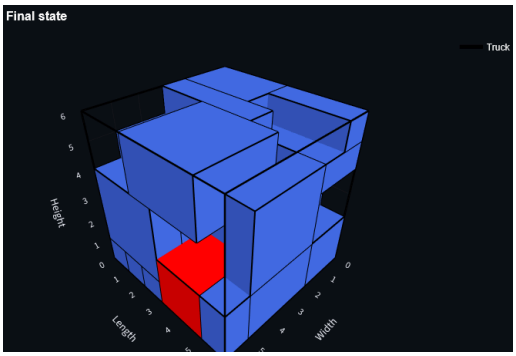
1. **Population size as control (67, {42, 100, 500}, 0.067)**
    a. **Test 1**

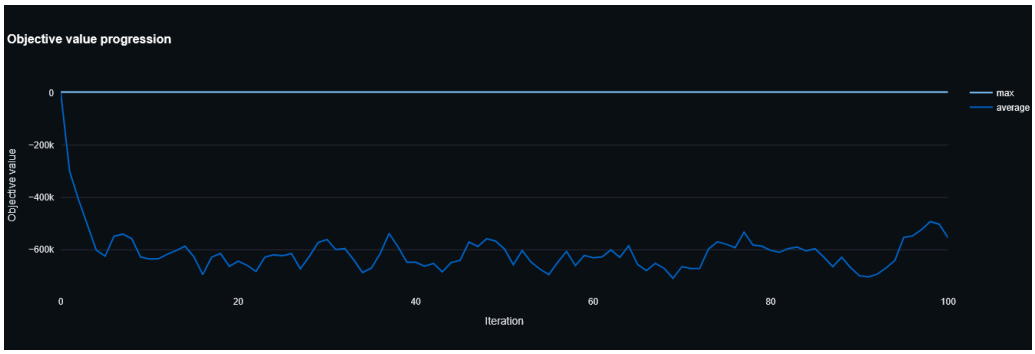
Initial State	Final State
	

Objective Function Plot


Iteration Count	42
Processing Time	17.0253s
Initial Value	3224
Final Value	3224

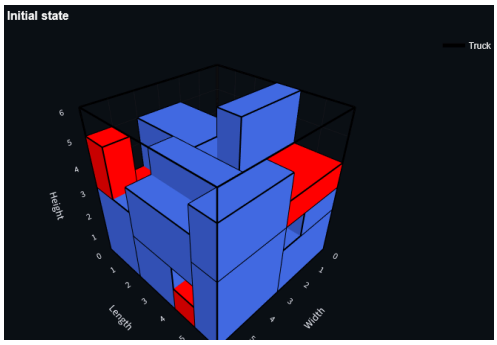
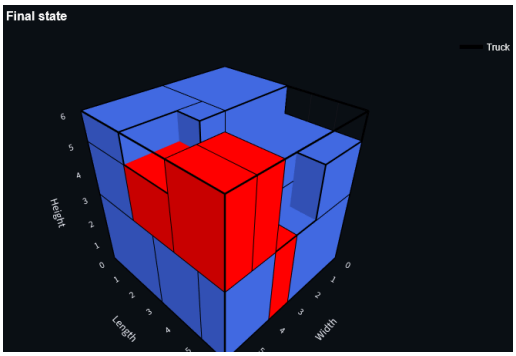
b. Test 2

Initial State	Final State
	
Objective Function Plot	

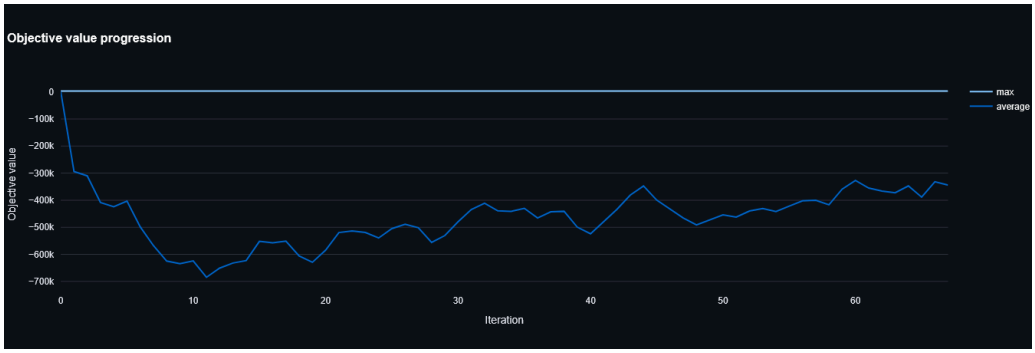


Iteration Count	100
Processing Time	31.5196s
Initial Value	3224
Final Value	3224

c. Test 3

Initial State	Final State
	

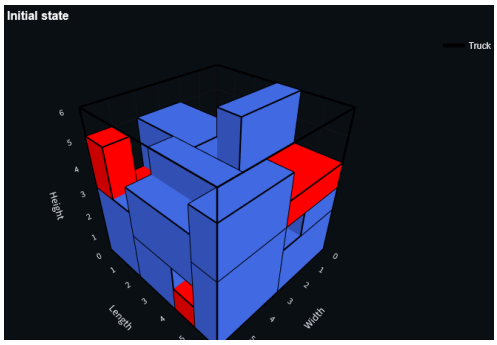
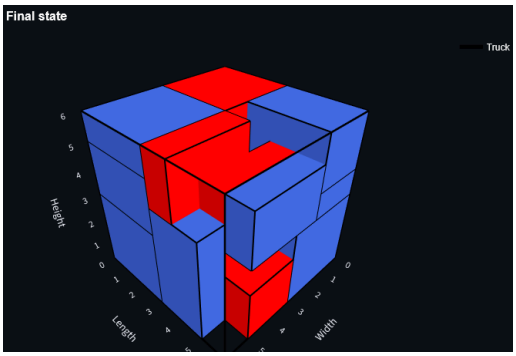
Objective Function Plot



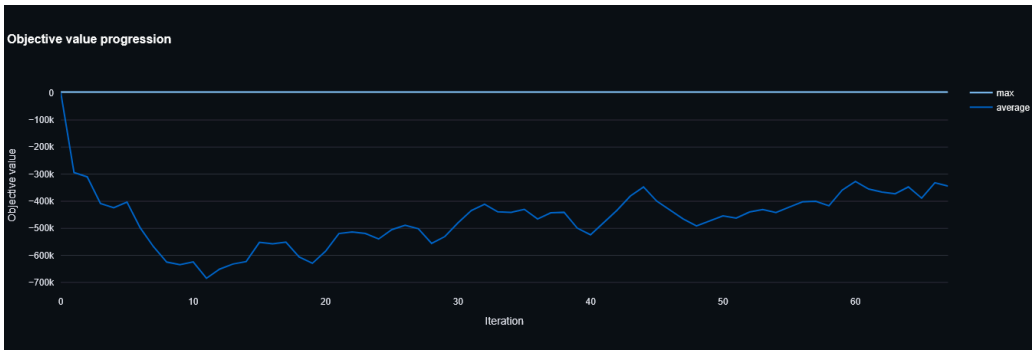
Iteration Count	3280
Processing Time	157.8839s
Initial Value	3224
Final Value	3280

2. Number of Generations as control ({42, 100, 500}, 67, 0.07)

a. Test 1

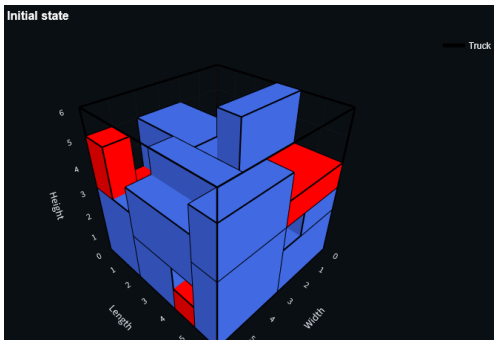
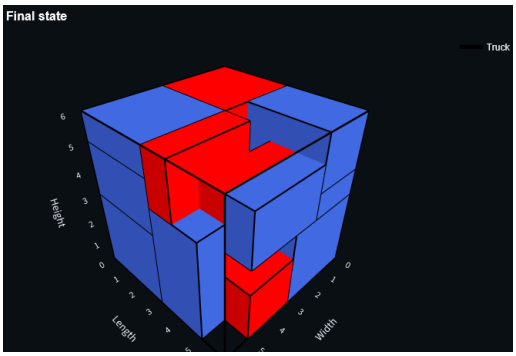
Initial State	Final State
	

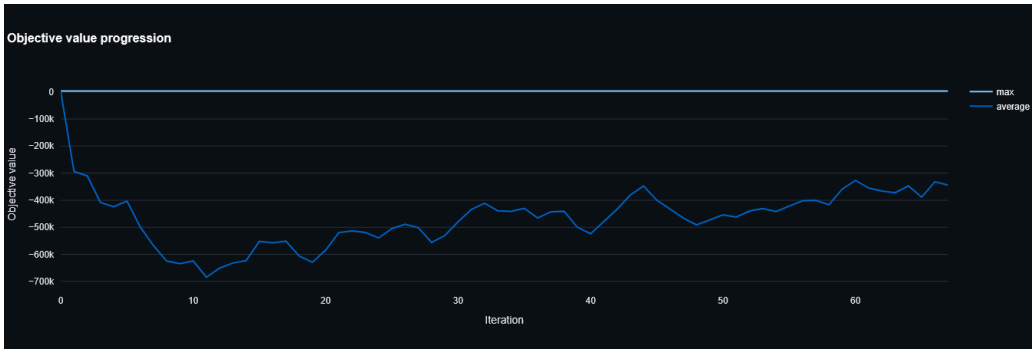
Objective Function Plot



Iteration Count	67
Processing Time	12.2482s
Initial Value	3085
Final Value	3143

b. Test 2

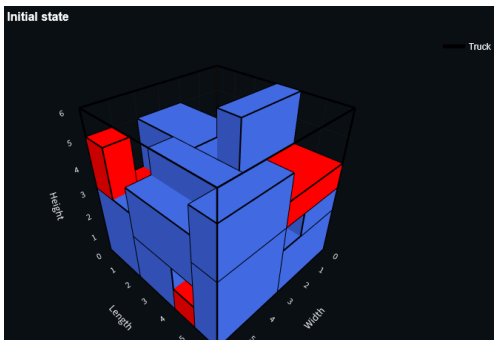
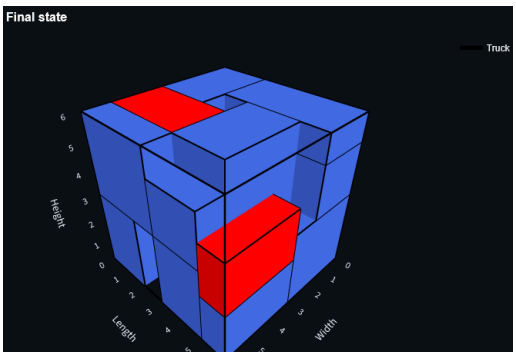
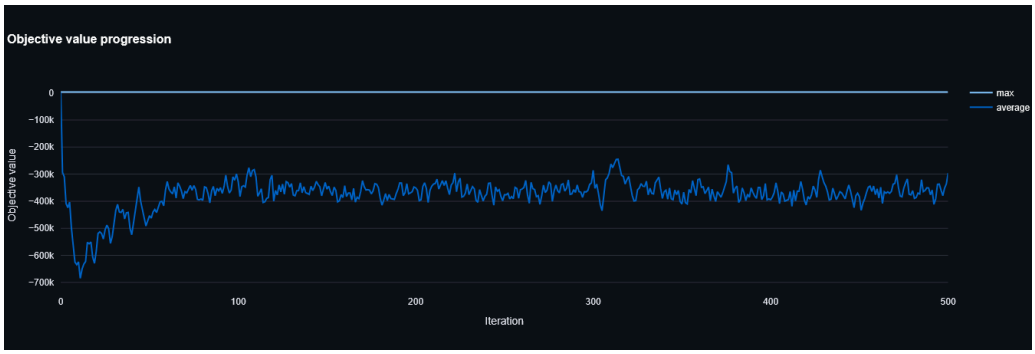
Initial State	Final State
	

Objective Function Plot


Iteration Count	100
Processing Time	20.2191s
Initial Value	3085
Final Value	3143

c. Test 3

Initial State	Final State

<table>
<tr><td> 
</td><td> 
</td></tr>
<tr><td colspan="2">Objective Function Plot</td></tr>
<tr><td colspan="2"> 
</td></tr>
<tr><td>Iteration Count</td><td>500</td></tr>
<tr><td>Processing Time</td><td>81.5009s</td></tr>
<tr><td>Initial Value</td><td>3085</td></tr>
<tr><td>Final Value</td><td>3143</td></tr>
</table>

Analisis

Solusi yang ditemukan oleh *Genetic Algorithm* mengalami kesulitan mencari *state* dengan nilai objektif yang lebih tinggi, dan memiliki rata-rata objective function value dari seluruh neighbor yang cenderung negatif. Jumlah iterasi dan populasi tidak jauh mempengaruhi hasil akhir.

Kesimpulan dan Saran

Penggunaan algoritma *local search* dalam persoalan *knapsack* dapat menghasilkan solusi optimum. Penggunaan jenis algoritma yang tepat memastikan solusi yang ditemukan merupakan solusi yang benar-benar terbaik dari semua kemungkinan konfigurasi solusi. Algoritma seperti Hill-Climbing dapat memberikan solusi yang cukup baik dalam waktu iterasi yang kecil, serupa dengan Simulated Annealing yang juga memberikan hasil yang tidak kalah baik.

Konfigurasi antara populasi dan iterasi untuk *Genetic Algorithm* tidak begitu mempengaruhi hasil akhir, dari ke-6 test yang dilakukan, tidak banyak perbedaan yang untuk hasil final state. Hal ini mungkin terjadi akibat kesalah implementasi.

Pembagian Tugas

NIM	Nama	Pembagian Tugas
13524072	I Gusti Ngurah Alit Dharma Yudha	Algoritma, Laporan, Testing
13524095	Jingglang Galih Rinenggan	Algoritma, Laporan, Testing
13524142	Rasyad Satyatma	Algoritma, Laporan, Testing

Referensi

[1] Tim Dosen Inteligensi Artifisial, “Beyond Classical Search”, Edunex, 2026. [Presentation]. Available: Edunex Document. [Accessed: Oct. 20, 2026]

Lampiran

Pernyataan AI Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027